

Mask R-CNN Segmentation Interview Modification Guide

How to adapt your current detection/segmentation template when an interview provides a Data folder

This guide summarizes the key parts you should know how to modify when adapting your current Mask R-CNN instance segmentation template. The uploaded code is based on PennFudanPed, reads PNGImages and PedMasks, and uses maskrcnn_resnet50_fpn.

1. First identify the type of segmentation task

Type	Goal	Common models	Fit for current template
Semantic Segmentation	Predict a class for every pixel, e.g., background / road / person	U-Net, FCN, DeepLabV3, SegFormer	Not ideal; use U-Net or DeepLabV3 instead
Instance Segmentation	Segment each object instance and output boxes / labels / masks / scores	Mask R-CNN	Very suitable

Your current template is an instance segmentation template because each target contains boxes, labels, masks, image_id, area, and iscrowd, and the model is Mask R-CNN.

2. Dataset path: the first thing to modify

If the interview provides a folder named Data, you normally should not download PennFudan. Replace the dataset path in main().

```
# Original PennFudan demo
DATA_ROOT = "data"
DATASET_NAME = "PennFudanPed"
```

```
# Interview dataset
DATASET_DIR = "Data"
dataset_dir = "Data"
```

In main(), replace:

```
dataset_dir = download_and_extract_dataset()
```

with:

```
dataset_dir = "Data"
```

3. Class names and NUM_CLASSES

For torchvision Mask R-CNN, class 0 must be background. Real object classes start from 1.

```
# Single foreground class
CLASS_NAMES = ["background", "pedestrian"]
NUM_CLASSES = len(CLASS_NAMES)
```

```
# Cat/dog instance segmentation
CLASS_NAMES = ["background", "cat", "dog"]
NUM_CLASSES = len(CLASS_NAMES)
```

Do not use CLASS_NAMES = ["cat", "dog"] because it misses the background class.

4. Image and mask folder names

Your current Dataset is hard-coded for PennFudan folder names:

```
self.imgs = sorted(os.listdir(os.path.join(root, "PNGImages")))
self.masks = sorted(os.listdir(os.path.join(root, "PedMasks")))

img_path = os.path.join(self.root, "PNGImages", self.imgs[idx])
mask_path = os.path.join(self.root, "PedMasks", self.masks[idx])
```

If the interview dataset uses Data/images and Data/masks, change them to:

```
self.imgs = sorted(os.listdir(os.path.join(root, "images")))
self.masks = sorted(os.listdir(os.path.join(root, "masks")))

img_path = os.path.join(self.root, "images", self.imgs[idx])
mask_path = os.path.join(self.root, "masks", self.masks[idx])
```

5. How to read masks

Case A: instance-id mask

If mask values are 0 for background and 1/2/3 for different instances, the current logic is suitable:

```
mask = np.array(Image.open(mask_path))
obj_ids = np.unique(mask)
obj_ids = obj_ids[1:] # remove background
masks = mask == obj_ids[:, None, None] # [N, H, W]
```

Case B: binary mask

If mask values are 0 for background and 255 for object, and there is only one foreground class, you can keep labels as class 1.

```
labels = torch.ones((boxes.shape[0],), dtype=torch.int64)
```

Case C: RGB color mask

If masks are color-coded, first convert RGB colors into class IDs.

```
mask_rgb = np.array(Image.open(mask_path).convert("RGB"))
class_mask = np.zeros(mask_rgb.shape[:2], dtype=np.uint8)

# red = cat, blue = dog
class_mask[(mask_rgb[:, :, 0] == 255) & (mask_rgb[:, :, 1] == 0) & (mask_rgb[:, :, 2] == 0)] = 1
class_mask[(mask_rgb[:, :, 0] == 0) & (mask_rgb[:, :, 1] == 0) & (mask_rgb[:, :, 2] == 255)] = 2

mask = class_mask
```

6. Label generation logic

The current template assumes every object is a pedestrian:

```
labels = torch.ones((boxes.shape[0],), dtype=torch.int64)
```

For multi-class segmentation, generate labels from mask IDs or annotation files. A simple label_map example is:

```
label_map = {"cat": 1, "dog": 2}
labels.append(label_map[class_name])
```

7. Replace both Mask R-CNN heads

Mask R-CNN requires replacing both the box predictor and the mask predictor.

```
def build_model(num_classes):
    model = maskrcnn_resnet50_fpn(weights="DEFAULT")

    # Replace box classification/regression head
    in_features_box = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features_box, num_classes)

    # Replace mask prediction head
    in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
    hidden_layer = 256
    model.roi_heads.mask_predictor = MaskRCNNPredictor(
        in_features_mask,
        hidden_layer,
        num_classes
    )
    return model
```

8. If the dataset already has train/val folders

The current template randomly splits the dataset. If Data already contains train and val folders, build separate datasets instead.

```
train_dataset = CustomSegmentationDataset(
    root=os.path.join("Data", "train"),
    transforms=get_transform(train=True)
)

val_dataset = CustomSegmentationDataset(
    root=os.path.join("Data", "val"),
    transforms=get_transform(train=False)
)
```

9. Recommended interview hyperparameters

Environment	BATCH_SIZE	NUM_EPOCHS	Notes
CPU quick demo	1	1	Only prove that the pipeline works; Mask R-CNN is slow on CPU
GPU interview	2	3	Train slightly longer, but still focus on running the full pipeline
Formal training	2 or larger	More epochs	Adjust based on GPU memory, dataset size, and time

10. Parts usually kept unchanged

- Keep `collate_fn` because each image may contain a different number of objects.
- Do not manually write a loss function. Torchvision Mask R-CNN returns a `loss_dict` during training.
- `run_inference` already handles boxes, labels, scores, and masks. Usually only `CLASS_NAMES` and thresholds need checking.

```
loss_dict = model(images, targets)
losses = sum(loss for loss in loss_dict.values())
```

11. Dataset inspection steps after receiving Data

```
print(os.listdir("Data"))
```

```
# Possible outputs:
# ['PNGImages', 'PedMasks']      -> current PennFudan-style Dataset works
# ['images', 'masks']           -> modify folder names in Dataset
# ['train', 'val']              -> build train_dataset and val_dataset separately
```

12. Interview plan you can say

I will treat this as an instance segmentation problem and use Mask R-CNN. First, I will inspect the dataset structure and check whether the annotations are instance masks, semantic masks, or COCO/polygon-style annotations. Then I will implement a custom PyTorch Dataset that returns each image and a target dictionary containing boxes, labels, masks, `image_id`, `area`, and `iscrowd`. For Mask R-CNN, class 0 is background, so I will define class names accordingly. I will load a pretrained Mask R-CNN model and replace both the box predictor and mask predictor for the number of target classes. During training, torchvision Mask R-CNN returns a dictionary of losses, including classification, box regression, mask, objectness, and RPN losses. I will sum these losses, backpropagate, and evaluate the model by visualizing predicted masks and boxes.

13. Final modification checklist

- `DATA_ROOT / dataset_dir`: change to the provided Data path.
- Dataset image/mask folders: `PNGImages/PedMasks` or `images/masks`.
- Mask reading mode: instance-id mask, binary mask, or RGB color mask.
- `CLASS_NAMES` must include background.

- NUM_CLASSES = len(CLASS_NAMES).
- Label generation: single class uses all ones; multi-class uses label_map.
- If train/val already exist, do not randomly split.
- For CPU demo, use BATCH_SIZE=1 and NUM_EPOCHS=1.
- In build_model, replace both box predictor and mask predictor.
- In run_inference, check score_threshold, mask_threshold, and CLASS_NAMES.